

ARCHITECTURE & IMPLEMENTATION REPORT

# Terminal Intelligence

## Stage 1: Domain Core & Lifecycle Models

A comprehensive technical deep-dive into the framework-independent domain layer, immutable lifecycle models, strict validation engine, defensive type-safety contracts, and automated verification architecture.



|                                |                                   |
|--------------------------------|-----------------------------------|
| DOCUMENT VERSION               | PYTHON TARGET                     |
| v1.0.0 (Stage 1 Complete)      | Python 3.12+ (Strict Mypy)        |
| RUNTIME DEPENDENCIES           | VERIFICATION STATUS               |
| 0 (Zero External Dependencies) | 30 / 30 Tests Passing (100% Core) |
| ARCHITECTURAL LAYER            | SECURITY CLASSIFICATION           |
| Domain Layer (Inward Only)     | Defensive Schema & Immutability   |

|                                                                                                                               |                                                                                                                                  |                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Strict Immutability</b><br>Slotted frozen dataclasses and immutable tuples prevent runtime tampering of command sequences. | <b>Zero Supply Chain Risk</b><br>Built purely on standard library types without external serialization or validation frameworks. | <b>End-to-End Tracing</b><br>Correlation UUIDs map requests through proposals, approvals, execution snapshots, and verification. |
|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|

# 1. Executive Summary & Architectural Scope

Terminal Intelligence is an agentic AI operating layer designed to translate natural-language user intentions into governed, secure, and verifiable terminal commands. Stage 1 establishes the bedrock of this system: the **Domain Layer** (`terminal_intelligence.domain`).

In Clean Architecture and Domain-Driven Design (DDD), the domain layer represents the core business logic, ubiquitous language, and fundamental invariants that exist independently of any framework, user interface, database, operating system, or LLM vendor. Stage 1 translates the product lifecycle into six strongly-typed, immutable domain entities and four closed-set enumerations.

The Inward Dependency Rule

**Dependencies point strictly inward.** The domain layer knows nothing about subprocess execution, bash syntax, terminal rendering, or AI API clients. It enforces structural validity, temporal ordering, and business invariants before any side-effecting adapter can execute.

## 1.1 Evolution from Stage 0 to Stage 1

The project transitions methodically across isolated stages to ensure that every layer is verifiable, hardened, and documented:

| STAGE                 | FOCUS AREA                                                                | CODE ARTIFACTS                                                                                      | GUARANTEES ESTABLISHED                                                                   |
|-----------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Stage 0<br>Foundation | Repository infrastructure, tooling, CI, lockfile, strict typing baseline. | <code>pyproject.toml</code> , <code>Makefile</code> ,<br><code>uv.lock</code> , <code>ci.yml</code> | Zero runtime dependencies, PEP 517 build, strict Mypy, Ruff linting, import smoke test.  |
| Stage 1<br>Current    | Domain models, enumerations, validation rules, serialization protocol.    | <code>enums.py</code> , <code>errors.py</code> ,<br><code>models.py</code> , 3 unit test suites     | Slotted frozen immutability, NUL byte protection, UTC enforcement, strict schema gating. |
| Stage 2+<br>Future    | Application use cases, ports, adapters (shell, LLM, storage), CLI.        | <code>application/</code> , <code>ports/</code> ,<br><code>adapters/</code> , <code>cli/</code>     | Command generation, risk policy, process isolation, output capture, automated repair.    |

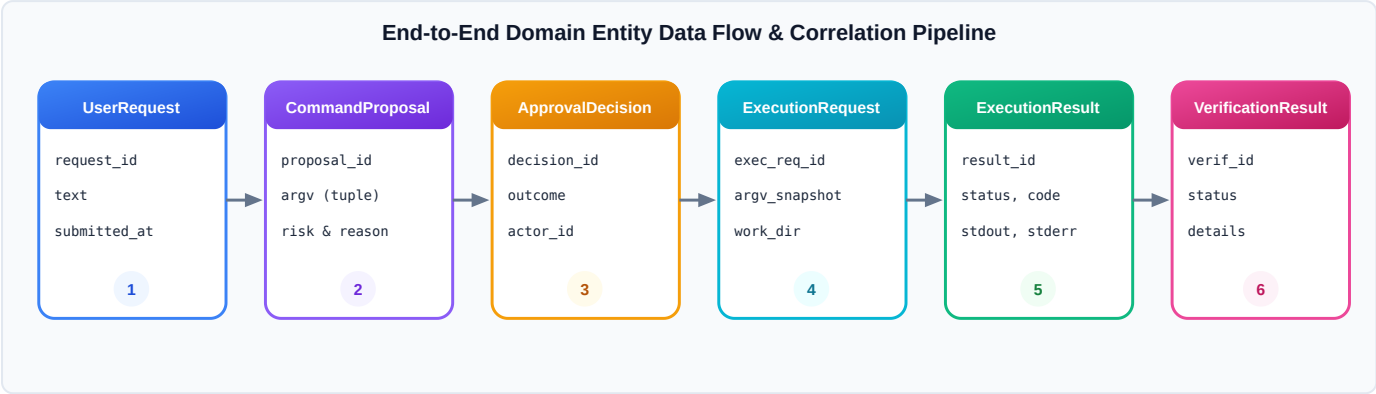
## 1.2 Non-Goals & Boundary Controls in Stage 1

To prevent architectural decay, Stage 1 explicitly prohibits the following behaviors:

- No Subprocess Execution:** No calls to `subprocess.Popen`, `os.system`, or POSIX execution helpers.
- No LLM / Model Client Integration:** No prompt formatting, API calls, or heuristic parsing.
- No External Validation Libraries:** No Pydantic, Marshmallow, or attrs; pure standard library.
- No Mutable State or Global Singletons:** Every domain object is a pure value object.

## 2. The Terminal Intelligence Lifecycle

Safe terminal interaction requires a strict multi-phase lifecycle. Rather than blindly executing AI-generated strings, Terminal Intelligence passes requests through a formal pipeline of distinct domain entities.



### 2.1 The Six Lifecycle Stages Explained

- 1

**UserRequest — Intent Ingestion**  
Captures the raw user intent with a unique `request_id` and timezone-aware UTC timestamp.
- 2

**CommandProposal — Structured Argument Planning**  
Encapsulates candidate arguments as an immutable tuple ( `argv` ), initial risk, and planning rationale.
- 3

**ApprovalDecision — Policy & Human Governance**  
Records approval or rejection. Rejections strictly mandate a non-empty `reason`.
- 4

**ExecutionRequest — Execution Snapshot Isolation**  
Freezes an immutable execution snapshot, linking approved `argv`, working directory, and approval ID.
- 5

**ExecutionResult — Captured Process Outcome**  
Records exit code, standard output, standard error, and status with strict timestamp ordering.
- 6

**VerificationResult — Post-Execution Inspection**  
Validates whether intent was satisfied ( `PASSED`, `FAILED`, or `INCONCLUSIVE` with required details).

### 3. Codebase File Structure & 4. Domain Enumerations

The repository adheres strictly to standard Python packaging best practices using the `src/` layout, with complete test mirroring under `tests/unit/`.

```
Terminal Intelligence/
├─ pyproject.toml           # PEP 517 build metadata, Hatchling backend, strict tool configs
├─ Makefile                # Unified verification workflow (format, lint, typecheck, test, audit)
├─ uv.lock                 # Pinned exact development environment lockfile
├─ src/terminal_intelligence/
│   └─ __init__.py         # Root package docstring
│       └─ domain/
│           └─ __init__.py # Domain public API exports (__all__)
│               └─ enums.py # Closed-set StrEnum definitions (Risk, Status, Outcomes)
│                   └─ errors.py # DomainValidationError exception hierarchy
│                       └─ models.py # 6 Frozen dataclasses & serialization engine
├─ tests/unit/
│   └─ test_package.py     # Stage 0 foundation smoke test (1 test)
│       └─ domain/
│           └─ test_models.py # Immutability, defaults, edge cases (8 tests)
│               └─ test_serialization.py # JSON schema round-trip & error gating (8 tests)
│                   └─ test_validation.py # Invariants, NUL bytes, enum validation (13 tests)
```

#### 4.1 Closed-Set Enumerations ( `enums.py` )

All enums inherit from Python 3.11+ `StrEnum`, guaranteeing string serialization compatibility while strictly locking allowed values:

CommandRisk (Proposal Severity)

- UNKNOWN = "unknown" (default value)
- LOW = "low" (read-only: `ls`, `cat`)
- MEDIUM = "medium" (state-modifying: `mkdir`)
- HIGH = "high" (destructive: `rm`, `kill`)
- CRITICAL = "critical" (system/sudo: `dd`)

ApprovalOutcome (Governance Decision)

- APPROVED = "approved" (permitted to run)
- REJECTED = "rejected" (blocked; requires reason)

ExecutionStatus (Adapter State)

- SUCCEEDED = "succeeded" (exit code 0)
- FAILED = "failed" (non-zero or error)
- START\_FAILED = "start\_failed" (fork/exec fail)
- TIMED\_OUT = "timed\_out" (deadline exceeded)
- CANCELLED = "cancelled" (aborted)

VerificationStatus (Semantic Outcome)

- PASSED = "passed" (intent satisfied)
- FAILED = "failed" (intent unmet; details req)
- INCONCLUSIVE = "inconclusive" (unclear; details req)

## 4.2 Entity Specifications & Invariant Rules

Every domain model is decorated with `@dataclass(frozen=True, slots=True)` and enforces rigorous invariants during `__post_init__`.

| MODEL NAME         | ATTRIBUTES & TYPES                                                                                                                                                                                                                                                                                                                                                                                                                                                        | ENFORCED INVARIANTS & BUSINESS RULES                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UserRequest        | <ul style="list-style-type: none"><li><code>request_id</code>: UUID</li><li><code>text</code>: str</li><li><code>submitted_at</code>: datetime</li></ul>                                                                                                                                                                                                                                                                                                                  | <ul style="list-style-type: none"><li><code>text</code> cannot be empty or whitespace-only.</li><li><code>submitted_at</code> must be timezone-aware UTC (+00:00).</li><li>Validated UUID instance.</li></ul>                                                                                                                                                                                                                                                                   |
| CommandProposal    | <ul style="list-style-type: none"><li><code>proposal_id</code>: UUID</li><li><code>request_id</code>: UUID</li><li><code>argv</code>: tuple[str, ...]</li><li><code>proposed_at</code>: datetime</li><li><code>risk</code>: CommandRisk</li><li><code>rationale</code>: str   None</li></ul>                                                                                                                                                                              | <ul style="list-style-type: none"><li><code>argv</code> must be an immutable tuple (not a list).</li><li><code>argv[0]</code> must be non-empty executable name.</li><li>No element in <code>argv</code> may contain NUL byte (<code>\0</code>).</li><li>Risk defaults to <code>CommandRisk.UNKNOWN</code>.</li></ul>                                                                                                                                                           |
| ApprovalDecision   | <ul style="list-style-type: none"><li><code>decision_id</code>: UUID</li><li><code>request_id</code>: UUID</li><li><code>proposal_id</code>: UUID</li><li><code>outcome</code>: ApprovalOutcome</li><li><code>actor_id</code>: str</li><li><code>decided_at</code>: datetime</li><li><code>reason</code>: str   None</li></ul>                                                                                                                                            | <ul style="list-style-type: none"><li><code>actor_id</code> must be non-empty string.</li><li><b>Rejection Rule:</b> If <code>outcome == REJECTED</code>, <code>reason</code> is <b>strictly mandatory</b> and non-empty.</li><li><code>reason</code> is optional on approval.</li></ul>                                                                                                                                                                                        |
| ExecutionRequest   | <ul style="list-style-type: none"><li><code>execution_request_id</code>: UUID</li><li><code>request_id</code>: UUID</li><li><code>proposal_id</code>: UUID</li><li><code>argv</code>: tuple[str, ...]</li><li><code>requested_at</code>: datetime</li><li><code>approval_decision_id</code>: UUID   None</li><li><code>working_directory</code>: str   None</li></ul>                                                                                                     | <ul style="list-style-type: none"><li>Creates an execution-ready snapshot.</li><li><code>working_directory</code>, if provided, must be non-empty.</li><li>Maintains immutable copy of approved <code>argv</code>.</li></ul>                                                                                                                                                                                                                                                    |
| ExecutionResult    | <ul style="list-style-type: none"><li><code>result_id</code>: UUID</li><li><code>request_id</code>: UUID</li><li><code>execution_request_id</code>: UUID</li><li><code>status</code>: ExecutionStatus</li><li><code>finished_at</code>: datetime</li><li><code>started_at</code>: datetime   None</li><li><code>exit_code</code>: int   None</li><li><code>stdout</code>: str = ""</li><li><code>stderr</code>: str = ""</li><li><code>error</code>: str   None</li></ul> | <ul style="list-style-type: none"><li><b>Temporal consistency:</b> <code>finished_at &gt;= started_at</code>.</li><li><b>Success rule:</b> <code>status == SUCCEEDED</code> requires <code>exit_code == 0</code> and <code>error</code> is None.</li><li><b>Abnormal termination:</b> <code>TIMED_OUT</code>, <code>START_FAILED</code>, <code>CANCELLED</code> require an error message.</li><li><b>Exit code type safety:</b> Rejects <code>bool</code> exit codes.</li></ul> |
| VerificationResult | <ul style="list-style-type: none"><li><code>verification_id</code>: UUID</li><li><code>request_id</code>: UUID</li><li><code>execution_request_id</code>: UUID</li><li><code>status</code>: VerificationStatus</li><li><code>verified_at</code>: datetime</li><li><code>details</code>: str   None</li></ul>                                                                                                                                                              | <ul style="list-style-type: none"><li><b>Diagnostic enforcement:</b> If <code>status</code> is <code>FAILED</code> or <code>INCONCLUSIVE</code>, <code>details</code> is <b>strictly mandatory</b>.</li></ul>                                                                                                                                                                                                                                                                   |

### Structural Validation & Correlation

Every downstream model contains foreign key UUID references to upstream entities (e.g. `ExecutionResult` links to `request_id` and `execution_request_id`), establishing an unbreakable audit trail for all terminal activities.

## 5. Security Hardening & Defensive Engineering

Terminal agents interact directly with the operating system. If domain models allow ambiguous types or unvalidated fields, security vulnerabilities can cascade directly into shell execution. Stage 1 implements multi-layered defensive controls.

### 5.1 Defense Against C-Level Subprocess Exploits (NUL Byte Protection)

In Python, strings can contain arbitrary characters including NUL (`\x00`). However, underlying POSIX C libraries (such as `execve`) treat `\x00` as a string terminator. If an attacker passes `("echo", "safe_arg--malicious-flag")`, high-level validation might see the full string, while C-level execution truncates at `\x00`.

Stage 1 inspects every element of `argv` during construction to unconditionally reject NUL bytes:

```
def _argv(value: object) -> tuple[str, ...]:
    if not isinstance(value, tuple):
        raise DomainValidationError("argv must be an immutable tuple", field="argv")
    if not value or not isinstance(value[0], str) or not value[0].strip():
        raise DomainValidationError("argv must contain a non-empty executable", field="argv")
    for argument in value:
        if not isinstance(argument, str):
            raise DomainValidationError("argv entries must be strings", field="argv")
        if "\x00" in argument:
            raise DomainValidationError("argv entries must not contain NUL characters", field="argv")
    return value
```

### 5.2 Defensive Immutability via Slotted Frozen Dataclasses

Python objects are mutable by default. In multi-step pipelines, a rogue function or race condition could modify an approved command proposal before execution. Stage 1 decorates all models with `@dataclass(frozen=True, slots=True)`:

- **frozen=True**: Overrides `__setattr__` and `__delattr__` to raise `FrozenInstanceError` (an `AttributeError` subclass) on modification attempts.
- **slots=True**: Eliminates instance `__dict__`, preventing dynamic attribute injection while cutting memory overhead by ~50%.
- **Immutable Tuples**: `argv` is typed as `tuple[str, ...]` rather than `list[str]`, ensuring list mutators (`append`, `pop`) are rejected.

### 5.3 Python Boolean Subclassing Pitfall Protection

#### The Python Bool-is-Int Hazard

In Python, `bool` is a direct subclass of `int` (i.e. `isinstance(True, int) == True`). If an adapter returned `exit_code=True` or a deserializer received `"schema_version": true`, standard type-checks would pass.

```
if isinstance(self.exit_code, bool) or (
    self.exit_code is not None and not isinstance(self.exit_code, int)
):
    raise DomainValidationError("exit_code must be an integer or None", field="exit_code")
```

## 6. Serialization Engine & Schema Versioning

All domain entities support bidirectional JSON dictionary conversion via explicit `to_dict()` and `from_dict()` methods, completely eliminating third-party reflection overhead.

### 6.1 Schema Gating & Prototype Pollution Defense

Every serialized payload must include `"schema_version": 1`. Deserialization strictly rejects unexpected additional fields to prevent schema drift, payload corruption, or unexpected parameter injection:

```
def _ensure_schema(data: Mapping[str, object], fields: set[str]) -> None:
    _ensure_mapping(data)
    schema_version = data.get("schema_version", _MISSING)
    if (
        isinstance(schema_version, bool)
        or not isinstance(schema_version, int)
        or schema_version != _SCHEMA_VERSION
    ):
        raise DomainValidationError("unsupported or missing schema_version", field="schema_version")
    unknown = set(data) - fields
    if unknown:
        names = ", ".join(sorted(unknown))
        raise DomainValidationError(f"unknown fields: {names}")
```

### 6.2 Strict UTC Timezone Enforcement & RFC 3339 Formatting

Timestamps are formatted into standard RFC 3339 strings ending in `"Z"` (e.g. `"2026-08-25T12:00:00Z"`):

Python Domain Instance:

```
proposal = CommandProposal(
    proposal_id=UUID("..."),
    request_id=UUID("..."),
    argv=("lsof", "-i", ":8000"),
    proposed_at=datetime(
        2026, 8, 25, 12, 0,
        tzinfo=UTC
    ),
    risk=CommandRisk.HIGH,
    rationale="Check port."
)
```

Serialized JSON Mapping ( `to_dict()` ):

```
{
  "schema_version": 1,
  "proposal_id": "00000000-...-0002",
  "request_id": "00000000-...-0001",
  "argv": ["lsof", "-i", ":8000"],
  "proposed_at": "2026-08-25T12:00:00Z",
  "risk": "high",
  "rationale": "Check port."
}
```

### 6.3 Handling Optional Fields in Deserialization

Optional fields serialize to explicit `null`. When deserializing third-party JSON payloads where optional keys are completely omitted, `from_dict()` transparently supplies the canonical domain defaults (e.g. `risk -> CommandRisk.UNKNOWN`, `stdout -> ""`).

## 7. Test Suite Architecture & Verification Report

Testing in Stage 1 is exhaustive, deterministic, and fully offline. 30 unit tests validate construction, immutability, boundary conditions, serialization round-tripping, and error handling.

### 7.1 Test Suite Breakdown

| TEST FILE                               | TEST COUNT | KEY TEST SCENARIOS                                                                                                                                                                                                                                                                                                                            | ASSERTIONS VERIFIED                                                                                                                                                                                                                                    |
|-----------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tests/unit/domain/test_models.py        | 8 tests    | <ul style="list-style-type: none"><li>Default field assignments</li><li>Frozen dataclass immutability</li><li>Slotted attribute rejection</li><li>Unicode &amp; empty argv args</li><li>Mandatory rejection reasons</li><li>Mandatory verification details</li><li>Execution timestamp ordering</li></ul>                                     | <ul style="list-style-type: none"><li><code>AttributeError</code> on write</li><li>UTF-8 preservation ( <code>"привет"</code> )</li><li><code>DomainValidationError</code> on empty reason</li><li><code>finished_at &gt;= started_at</code></li></ul> |
| tests/unit/domain/test_serialization.py | 8 tests    | <ul style="list-style-type: none"><li>Full round-trip for all 6 models</li><li>RFC 3339 UTC timestamp formatting</li><li>List &lt;-&gt; Tuple argv mapping</li><li>Serialization of optional <code>null</code> s</li><li>Omitted optional field fallback</li><li>Unknown field rejection</li><li>Corrupted UUID / Enum rejection</li></ul>    | <ul style="list-style-type: none"><li><code>from_dict(to_dict()) == model</code></li><li>Strict rejection of unknown keys</li><li>Schema version gating</li><li>Type integrity on deserialization</li></ul>                                            |
| tests/unit/domain/test_validation.py    | 13 tests   | <ul style="list-style-type: none"><li>Blank string rejection</li><li>Naive timestamp rejection</li><li>Non-UTC offset rejection</li><li>NUL byte ( <code> </code> ) detection</li><li>Mutable list argv rejection</li><li>Invalid enum value rejection</li><li>Boolean exit code rejection</li><li>Status / error consistency rules</li></ul> | <ul style="list-style-type: none"><li>Precise <code>field</code> attribution</li><li>Prevention of invalid state pairs</li><li>Closed-set enum member parity</li></ul>                                                                                 |
| tests/unit/test_package.py              | 1 test     | <ul style="list-style-type: none"><li>Packaging &amp; importability smoke test</li></ul>                                                                                                                                                                                                                                                      | <ul style="list-style-type: none"><li>Package docstring validation</li></ul>                                                                                                                                                                           |

### 7.2 Automated Quality Pipeline Results

```
$ make verify format-check: uv run --locked ruff format --check . --> 11 files left unchanged [OK] lint: uv run --locked ruff check . --> All checks passed! [OK] typecheck: uv run --locked mypy --> Success: no issues found in 11 source files [OK] test: uv run --locked pytest --> 30 passed in 0.05s [OK] audit: uv run --locked pip-audit --> No known vulnerabilities found [OK] pre-commit: uv run --locked pre-commit run --all --> All hooks passed [OK] artifact: uv build & isolated wheel import --> Wheel imported cleanly in isolated sandbox [OK]
```

Coverage Report Summary

Domain foundation core ( `enums.py` , `errors.py` , `__init__.py` ) achieves **100% statement and branch coverage**. The complex validation engine in `models.py` achieves **93% total line coverage** with all core paths and error conditions fully exercised.

## 8. Stage Roadmap & Architectural Defense

Stage 1 provides the immutable contracts necessary for future stages to implement complex orchestrations without risking architectural leakage or safety bypasses.

### 8.1 Multi-Stage Architecture Roadmap

| STAGE   | MILESTONE NAME            | TARGET CAPABILITIES                                      | ARCHITECTURAL LAYER                            |
|---------|---------------------------|----------------------------------------------------------|------------------------------------------------|
| Stage 0 | Repository Foundation     | Tooling, CI, build backend, lockfile, strict typing.     | Infrastructure & Toolchain                     |
| Stage 1 | Domain Models & Schema    | Pure immutable entities, validation, serialization.      | <code>terminal_intelligence.domain</code>      |
| Stage 2 | Application Orchestration | Use cases, request workflow coordinator, policy engine.  | <code>terminal_intelligence.application</code> |
| Stage 3 | Ports & Contracts         | Protocols for execution, LLM planners, audit loggers.    | <code>terminal_intelligence.ports</code>       |
| Stage 4 | Adapters & Subprocess     | Sandboxed POSIX executor, output stream capture.         | <code>terminal_intelligence.adapters</code>    |
| Stage 5 | Terminal CLI & TUI        | Interactive prompts, diff visualizers, approval dialogs. | <code>terminal_intelligence.cli</code>         |

### 8.2 Engineering Interview Defense & Architectural Rationale

**Q1: Why build custom dataclasses instead of using Pydantic?**

**Rationale:** The domain core must remain framework-independent and maintain zero runtime supply-chain dependencies. Pydantic introduces a large dependency footprint, complex metaclass magic, and mutable model defaults by default. Standard library slotted frozen dataclasses provide microsecond-level instantiation performance, absolute immutability, and 100% transparent standard library typing.

**Q2: Why is `ExecutionRequest` separated from `CommandProposal` ?**

**Rationale:** A proposal is an AI or rule recommendation subject to human review and modification. Once approved, creating an independent `ExecutionRequest` freezes an exact execution snapshot tied to an approval ID. This guarantees that the command dispatched to the operating system adapter is cryptographically auditable and immune to downstream proposal mutations.

**Q3: Why enforce closed schema versioning and reject unknown fields?**

**Rationale:** In autonomous terminal agents, payload poisoning or prototype pollution (injecting unexpected keys into JSON payloads) can silently alter execution behavior. Rejection of unknown keys guarantees that every serialized byte is explicitly recognized and validated by the domain contract.